13. Optimizations

13.1 The running sums carry no labels

Section 6 wrote every running sum as "(A, p0) [2,0,1,1]". The pair (A, p0) is correct, but it is **not stored** next to the sum. The labels are needed only inside the PE, to choose *which* running sum to add to. After that, the place of the sum in the bundle already says whose it is.

(a) The bundle is a row of numbered slots

A group carries F × G = 2 × 2 = 4 running sums down the column, always in the same order: **slot = 2 · filter label + member label**. The member label is local to the group: member 0 of G1 is p0, member 0 of G2 is p2.

slot e	filter label = e / 2	member label = e % 2	member in G1	member in G2
0	0 (A)	0	p0	p2
1	0 (A)	1	p1	p3
2	1 (B)	0	p0	p2
3	1 (B)	1	p1	p3

(b) Inside the PE, the two labels pick the slot

The PE computes the slot number from its own filter label and the label on each spike bit; it reads that slot, adds its weight, and writes it back. Nothing is searched: the number is known the moment the bits arrive.

PE k2: weight 1, filter label 1 (B)	lane t0	lane t1	lane t2	lane t3
spike bit of G1	0	1	1	1
member label	-	0 (p0)	1 (p1)	1 (p1)
slot = 2 · filter label + member label	no spike	2·1 + 0 = **2** → +1	2·1 + 1 = **3** → +1	2·1 + 1 = **3** → +1

Same cycle as Figure 10: +1 to slot 2 (B, p0) at t1 and to slot 3 (B, p1) at t2 and t3.

(c) What leaves the bottom: bare sums, one touch bit each, and one small sideband per group

G1 leaves the column

slot 0	[2,0,1,1]	touch 1
slot 1	[0,0,0,0]	touch 0
slot 2	[0,1,0,0]	touch 1
slot 3	[0,0,4,1]	touch 1

sideband (once per group): members [p0, p1]
column filter list [A, B] (fixed for the pass)

G2 leaves the column

slot 0	[1,2,0,0]	touch 1
slot 1	[0,1,2,2]	touch 1
slot 2	[0,0,3,3]	touch 1
slot 3	[1,0,0,0]	touch 1

sideband (once per group): members [p2, p3]
column filter list [A, B] (fixed for the pass)

(d) The unit below the array turns slot numbers back into (filter, pixel)

For slot e: filter = filter list[e / 2], pixel = members[e % 2]. Since e is fixed for every slot, this is plain wiring, not arithmetic (in the RTL: `{fid[e / G], pid[e % G], sums}`).

G1 slot	bare sums	list[e/2]	members[e%2]	write to
0	[2,0,1,1]	A	p0	output[A][p0]
1	[0,0,0,0]	A	p1	skip (touch 0)
2	[0,1,0,0]	B	p0	output[B][p0]
3	[0,0,4,1]	B	p1	output[B][p1]

G2 slot	bare sums	list[e/2]	members[e%2]	write to
0	[1,2,0,0]	A	p2	output[A][p2]
1	[0,1,2,2]	A	p3	output[A][p3]
2	[0,0,3,3]	B	p2	output[B][p2]
3	[1,0,0,0]	B	p3	output[B][p3]

	p0	p1	p2	p3
A	[2,0,1,1]	[0,0,0,0]	[1,2,0,0]	[0,1,2,2]
B	[0,1,0,0]	[0,0,4,1]	[0,0,3,3]	[1,0,0,0]

Output after both groups: identical to the GEMM (checked for every entry). Slot 1 of G1, (A, p1), was never touched, so nothing is written and output[A][p1] stays [0,0,0,0].

> **Rule.** A label is needed to *choose* a place; once a value sits in a fixed place, the place is its label. The weights and the spike bits carry labels because a PE cannot tell their filter or pixel from where they are. The running sums sit in fixed slots, so they carry none; the only extra data is the group's member list, sent once per group beside the column.

(e) What this saves in the real design (4 × 4 array, 16 slots of T = 8 sums)

	flip-flops	where
Fixed slots + ID sideband (ours)	**1,144**	one 52-bit ID record per group, delayed R + c cycles beside each column (4 + 5 + 6 + 7 stages)
13-bit (filter, pixel) address stored on every slot	3,328	16 slots × 13 b in the bundle register of each of the 16 PEs

Flip-flop counts from the RTL widths; about 2.9× fewer, and no comparator in any PE.